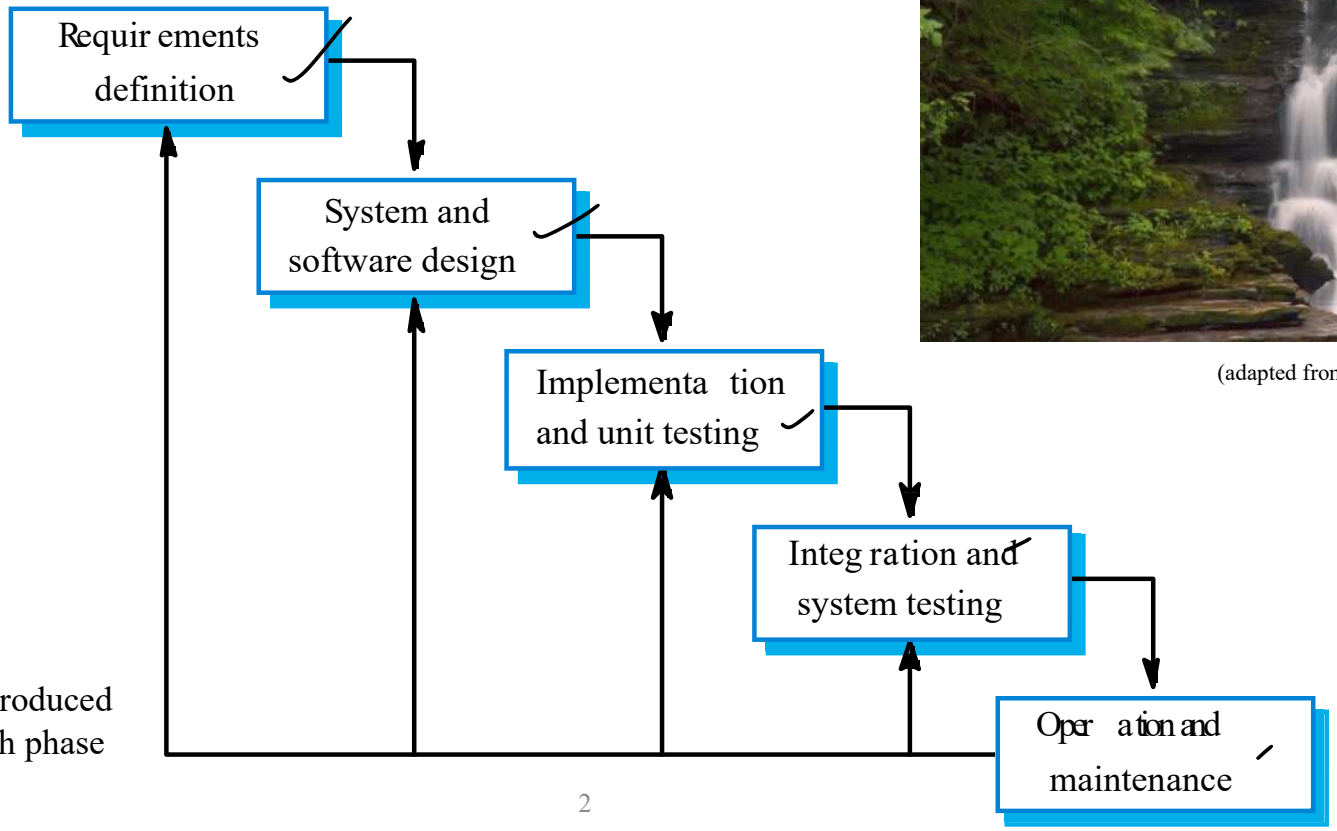
Week 9

Let’s review these basic concepts.

MITS 5400G Secure Software Systems

Winter 2026

Waterfall Model



Documents are produced at the end of each phase

(adapted from Google Images)

2

Waterfall Model Pros and Cons

- This model is only appropriate when
  - the requirements are well-understood and
  - changes will be fairly limited during the design process.
- Applicability
  - Large systems engineering projects where a system is developed at several sites.
- Cons
  - The partitioning into distinct stages makes it difficult to respond to changing customer requirements.
- Few business systems have stable requirements.

Requirements Engineering

- Requirements are:
  - Descriptions of the **services** provided by the system
  - Operational **constraints** of the system
- Defining <u>what</u> the system should do
  - What the customer needs (as opposed to wants)
  - Not <u>how</u> the solution should be designed or implemented (i.e., not a design document)
- Requirements engineering: three iterative activities
  - Elicitation: capturing information from sources
  - Documentation: "writing it on paper"
  - Validation: confirming whether it meets users' needs

EDV

Requirements Engineering (cont.)

- Where and how to gather requirements?
- Sources of requirements
  - Interviews, questionnaires with the customer
  - Existing/similar systems, standards in that domain, rapid prototypes
- Requirements document
  - Summary of software product vision
  - Key features list
  - List of details about end-user requirements
  - Exhaustive list of details about screens, Graphical User Interface (GUI), input/output conditions, all system states and state changes, all persistent data, etc.
    - Called the Software Requirements Specification (SRS) document

Example: Requirements Document

- Example: Online library system for a university
- Vision Statement
  - You are asked to develop an <u>online system for a university library</u>. It is an <u>interactive system</u> that support <u>bookkeeping tasks and user browsing</u> online.
- Feature list (for Books and Journals)
  - Library contains books and journals
  - Each book may have several copies
  - Some books are short-term loans, all others 3 weeks
  - Only staff can borrow journals
  - Members can borrow up to 6 items at a time
  - ……

Types of Requirement

- User requirements
  - Statements in natural language, diagrams of the services the system provides, its operational constraints
  - Written for customers
- System requirements
  - A structured document with detailed descriptions of the system's functions, services and operational constraints.
  - Defines what should be implemented; it could be part of a contract between client and contractor.

Definitions and Specifications

Natural language

- Example of **user requirement definition**
  - 1. The software must provide a means of representing and accessing external files created by other tools.

Detailed descriptions

- Example of **system requirements specification**
  - 1.1 The user should be provided with facilities to define the type of external files.
  - 1.2 Each external file type may have an associated tool which may be applied to the file.
  - 1.3 Each external file type may be represented as a specific icon on the user's display.
  - 1.4. Facilities should be provided for the icon representing an external file type to be defined by the user.
  - 1.5. When a user selects an icon representing an external file, the effect of that selection is to apply the tool associated with the type of the external file to the file represented by the selected icon.

Classification of Software System Requirements

- Functional requirements
  - **Services** that the system should provide, e.g., how the system should respond to particular inputs and situations.
- Non-functional requirements
  - **Constraints** on the services provided by the system.
  - Example: timing constraints, development process constraints, standards, etc.
- Domain requirements
  - Requirements that reflect characteristics of the **application domain** of the system.
  - They can be functional or non-functional.

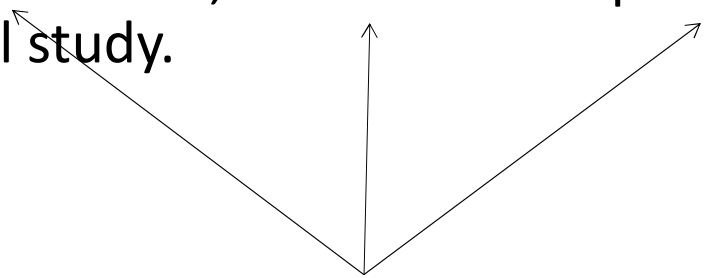
Functional Requirements

- Describe system services
- Depend on the type of software, expected users and the type of system for running the software
- Functional user requirements: high-level statements of what the system should do
- Functional system requirements: describe the system services in detail

Example: The LIBSYS System

- A library system that provides a single interface to a number of databases of articles in different libraries.

- Users can search for, download and print these articles for personal study.



CASE

Non-functional Requirements

- Define system properties: e.g., reliability, response time and storage requirements.
- Define constraints: e.g., I/O device capability, system representations, etc.
- Process requirements: e.g., development with a particular Computer-aided Software Engineering (CASE) system, programming language, development method.
- Non-functional requirements may be more critical than functional requirements.
  - If these are not met, the system is useless.

Non-functional Classifications

Type	Meaning	Example
Product requirements	Specify that the delivered product must behave in a particular way, e.g., execution speed, reliability, portability, and usability.	The user interface for LIBSYS shall be implemented as simple HTML without frames or Java applets.
Organisational requirements	A consequence of organisational policies and procedures, e.g. process standards used, implementation requirements, etc.	The system development process and deliverable documents shall conform to the process and deliverables defined in XYZCo-SP-STAN-95.
External requirements	Arise from factors which are external to the system and its development process, e.g., interoperability requirements, legislative requirements, etc.	The system shall not disclose any personal information about customers (except their names and reference numbers) to the operators of the system.

Domain Requirements

- Describe system characteristics that reflect the application domain.
- Domain requirements
  - New functional requirements
  - Constraints on existing requirements
  - Define specific computations
- If domain requirements are not satisfied, the system may not work correctly.

Example: The LIBSYS System

- There shall be a standard user interface to all databases which shall be based on the Z39.50 standard.

- Because of copyright restrictions, some documents must be deleted immediately on arrival. Depending on the user's requirements, these documents will either be printed locally on the system server for manually forwarding to the user or routed to a network printer.

Domain Requirements Problems

- Understandability
  - Requirements are expressed in the language of the application domain.
  - This is often not understood by software engineers developing the system.
- Implicitness
  - Domain specialists understand the area so well ➔ They do not explicitly think of domain requirements.

User Requirements

- Describe requirements in such a way that they are understandable by system users.
  - System users do not have detailed technical knowledge.
- User requirements are defined using natural language, tables and diagrams.
  - These can be understood by all users.

Problems with Natural Language (NL)

- Lack of clarity
  - Precision is difficult without making the document difficult to read
- Requirements confusion
  - Functional and non-functional requirements tend to be mixed-up
- Requirements amalgamation
  - Several different requirements may be expressed together

Example: Problems in LIBSYS Requirement

4.5 LIBSYS shall provide a financial accounting system that maintains records of all payments made by users of the system. System managers may configure this system so that regular users may receive discounted rates.

Describes the **concept** of a financial accounting system that is to be included in LIBSYS.

However, it also includes the **detail** that managers can configure this system - this is unnecessary at this level.

System Requirements

- More detailed specifications (of system services and constraints) than user requirements
- Form the basis for designing the system
- They may be incorporated into the system contract.
- System requirements may be defined by using system models.

Structured Language Specifications

- Use a predefined template to limit the freedom of the requirements writer.
- All requirements are written in a standard way (e.g., using a form or a table).
- The terminology used in the description may be limited.
- Advantage
  - Most of the expressiveness of natural language is maintained.
  - A degree of uniformity is imposed on the specification.

Form-based Specification

Insulin Pump/Control Software/SRS/3.3.2

Function Compute insulin dose: Safe sugar level

Description Computes the dose of insulin to be delivered when the current measured sugar level is in the safe zone between 3 and 7 units.

Inputs Current sugar reading (r2), the previous two readings (r0 and r1)

Source Current sugar reading from sensor. Other readings from memory.

Outputs CompDose – the dose in insulin to be delivered

Destination Main control loop

Action: CompDose is zero if the sugar level is stable or falling or if the level is increasing but the rate of increase is decreasing. If the level is increasing and the rate of increase is increasing, then CompDose is computed by dividing the difference between the current sugar level and the previous level by 4 and rounding the result. If the result, is rounded to zero then CompDose is set to the minimum dose that can be delivered.

Requires Two previous readings so that the rate of change of sugar level can be computed.

Pre-condition The insulin reservoir contains at least the maximum allowed single dose of insulin..

Post-condition r0 is replaced by r1 then r1 is replaced by r2

Side-effects None

Tabular Specification

- Used to supplement natural language
- Useful for defining a number of possible actions

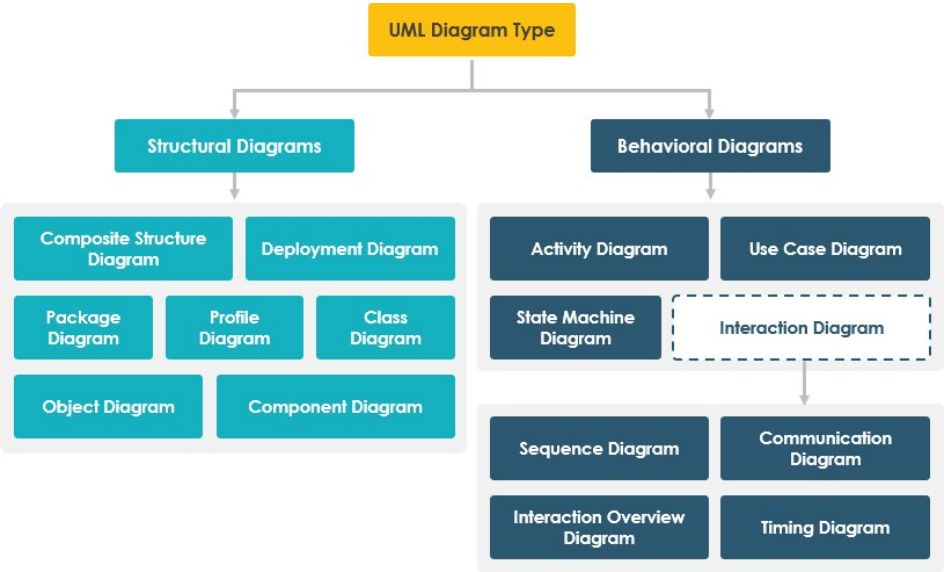
Condition	Action
Sugar level falling (r2 < r1)	CompDose = 0
Sugar level stable (r2 = r1)	CompDose = 0
Sugar level increasing and rate of increase decreasing ((r2-r1)<(r1-r0))	CompDose = 0
Sugar level increasing and rate of increase stable or increasing. ((r2-r1) ≥ (r1-r0))	CompDose = round ((r2-r1)/4) If rounded result = 0 then CompDose = MinimumDose

Graphical Models

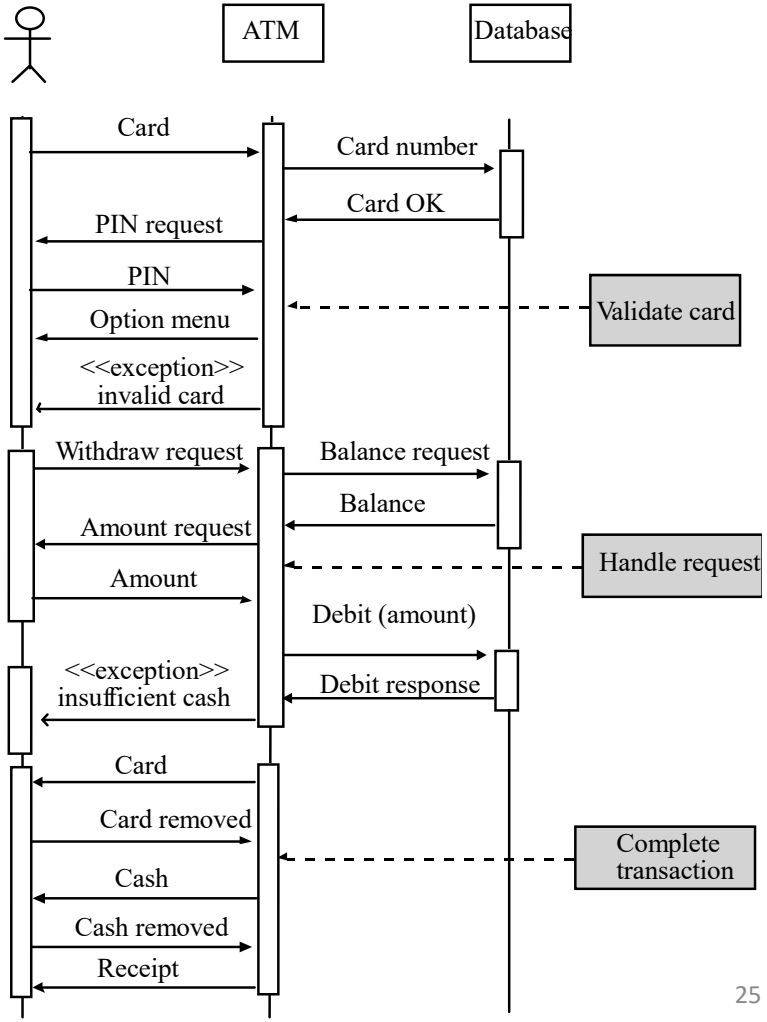
- Graphical models are useful for
  - Showing how state changes
  - Describing a sequence of actions

Unified Modelling Language (UML) - Sequence Diagram of ATM Withdrawal

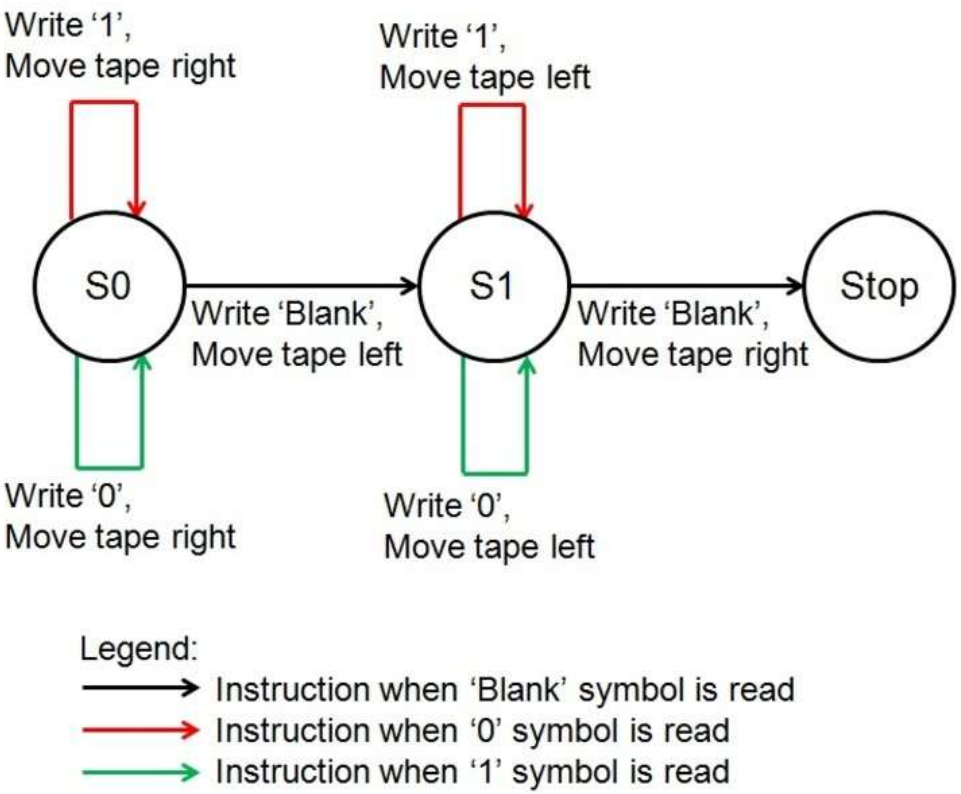
- Show the sequence of events that take place during user interaction with a system
- Order of actions: from top to bottom
- Cash withdrawal from an ATM
  - Validate card
  - Handle request
  - Complete transaction

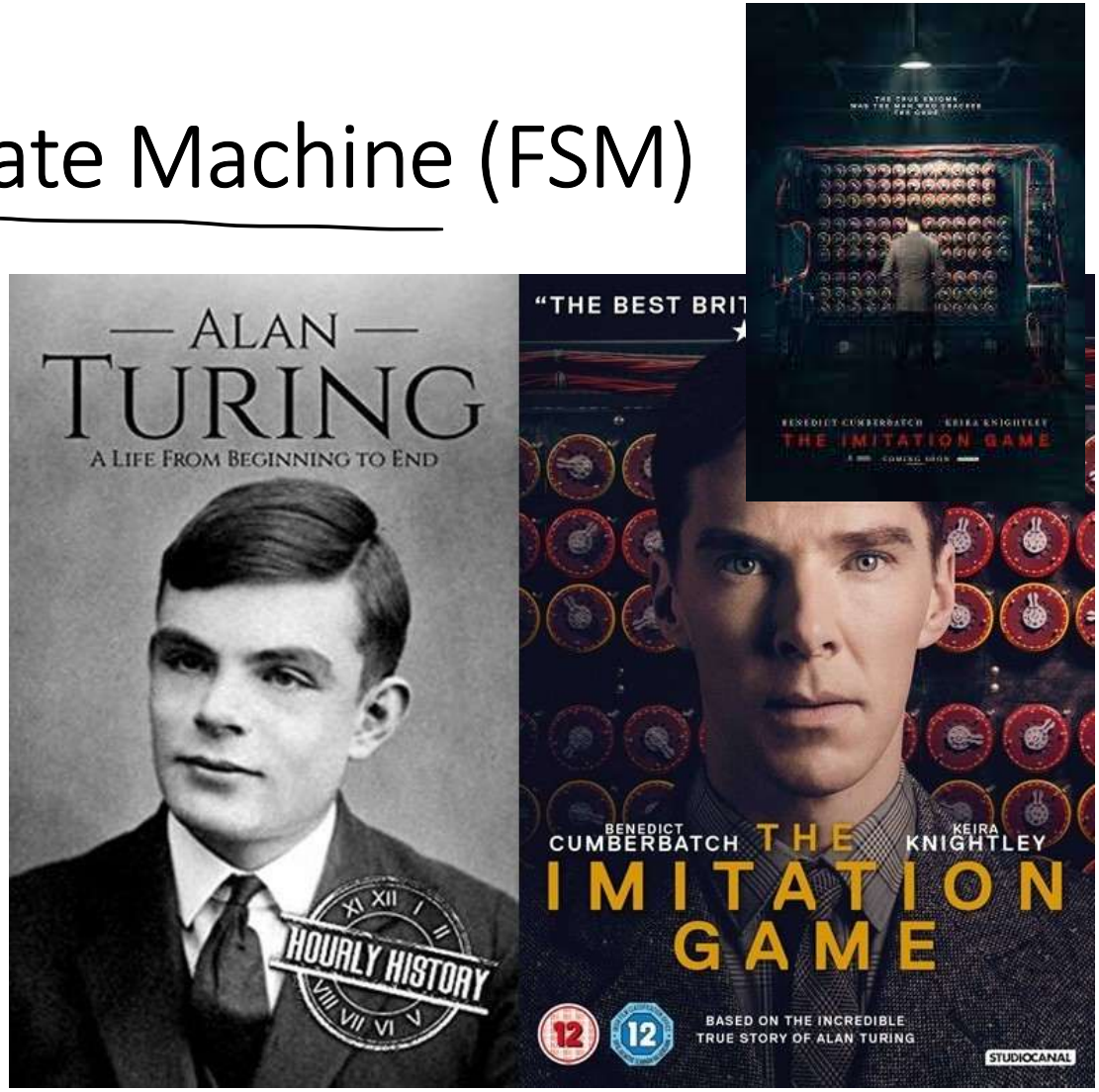


Source: Google Images



Alan Turing – Finite State Machine (FSM)





Source: Google Images